Duckietown Module 3

Modeling and Kinematics

Credits to Duckietown MOOC Creators for content in slides

States

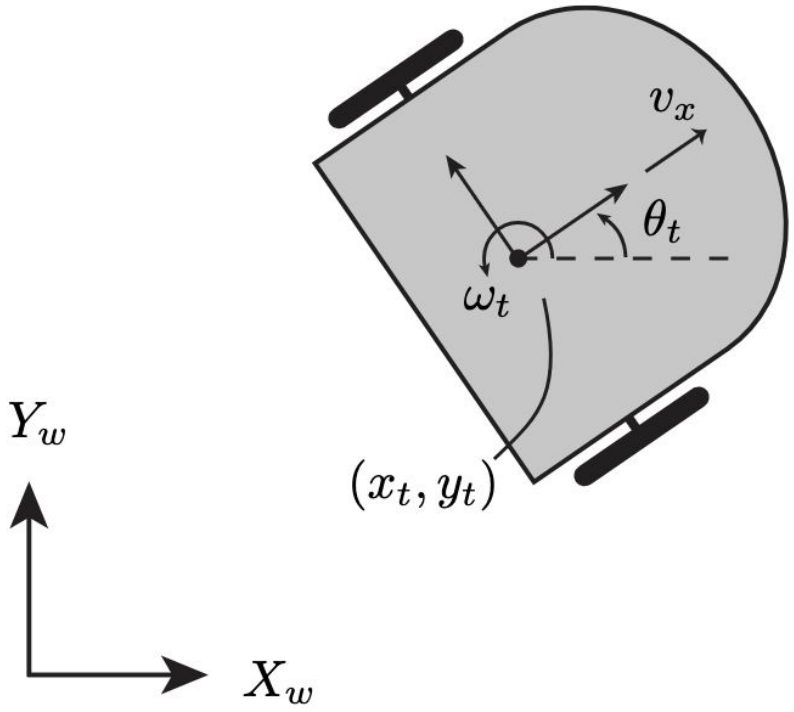
- Representation of the robot’s position
  - Relative to itself and its environment
- Observed and measured through robot’s sensors
- Represented through variable q

How does it relate to Duckietown?

- Pose + orientation of Duckiebot
- 2 dimensional environment + angle
- For all intents and purposes, Duckietown mainly cares about 2D representation

Duckiebot State

- $q = [x_t, y_t, \Theta_t]^T$
  - T = time
- 2 reference frames
  - World frame and robot frame
- x_r, y_r is the x and y coordinates of the robot from the robot's perspective
  - Θ is determined by the angle between x_A and x_r where A is the midpoint between the axles
- x_w, y_w is the x and y coordinates of the robot from the world's perspective

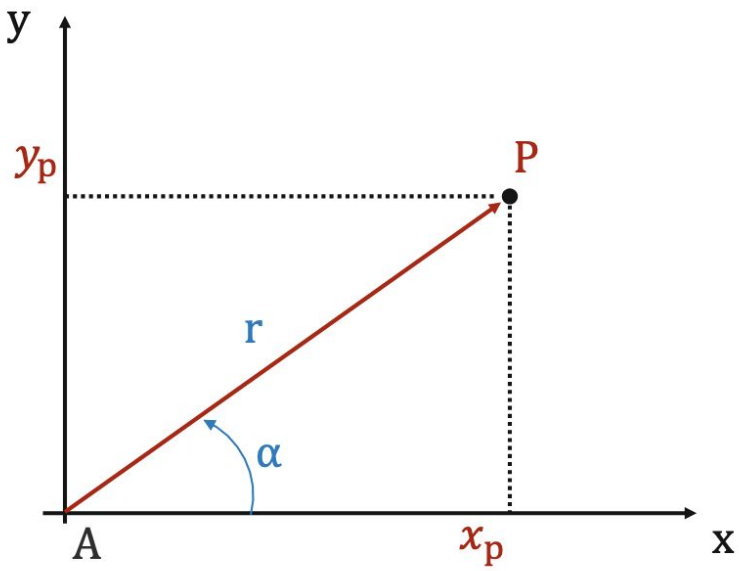


Types of Coordinates

- Cartesian
  - Standard x, y coordinates we see
- Polar
  - Uses R and α where R is magnitude and α is angle
  - Sometimes Θ is used instead to represent angle

Conversion between Cartesian and Polar

- For some point P represented as $P^A = \begin{bmatrix} x \\ y \end{bmatrix}$
- $P^A = r = re^{j\alpha} = (r, \alpha)$
- r is some magnitude
- α is an angle relative to A_x
- $x_p = r\cos\alpha$
- $y_p = r\sin\alpha$
- $r = \sqrt{(x_p^2 + y_p^2)}$
- $\alpha = \operatorname{atan}(y_p / x_p)$



Duckiebot Movement

- Rotations (Cartesian representation):

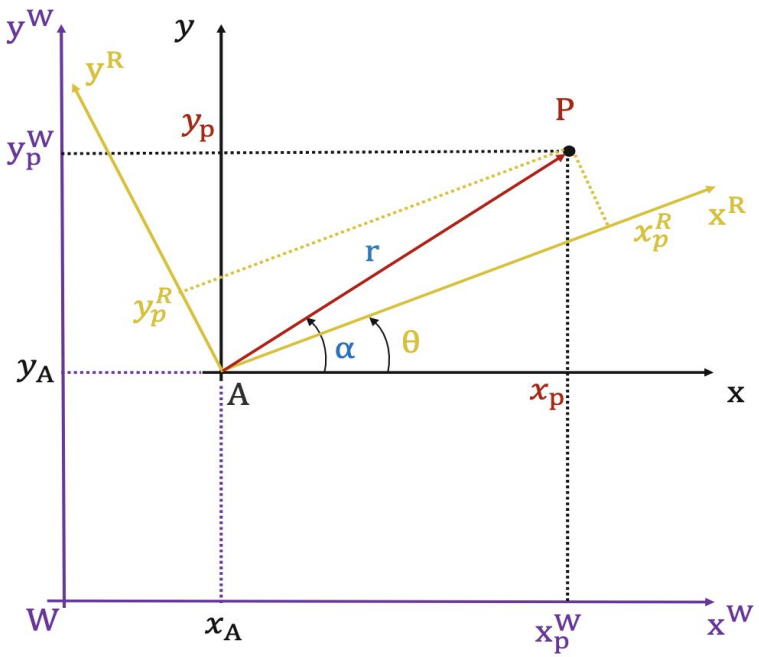
$$P^A = \begin{bmatrix} x_p \\ y_p \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_p^R \\ y_p^R \\ 1 \end{bmatrix} = \mathbf{R}(\theta) P^R$$

- Translations:

$$P^W := \begin{bmatrix} x_p^W \\ y_p^W \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & x_A \\ 0 & 1 & y_A \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_p \\ y_p \\ 1 \end{bmatrix} = \mathbf{T} P^A$$

- Rotations and Translations together:

$$P^W = \mathbf{T} P^A = \mathbf{T}\mathbf{R}(\theta) P^R = \begin{bmatrix} \cos\theta & -\sin\theta & x_a \\ \sin\theta & \cos\theta & y_a \\ 0 & 0 & 1 \end{bmatrix} P^R$$



We can now express coordinates of points in roto-translated reference systems w.r.t. to a “world” reference

Differential Drive

- Each wheel of the Duckiebot is operated by a different DC motor
- Two types of states to consider
  - Angular and Linear
- Pose of Duckiebot
  - Pose of the wheel
  - Position of the robot
- Motion of the Duckiebot
  - Angular velocity, robot velocity
  - Angular acceleration, robot acceleration

Linear Motion

- Recall from calculus derivatives
- x = some position in a linear environment
- dx/dt is where we take the derivative of x with respect to time
- v = dx/dt, which is velocity
  - How fast something is changing its position, a.k.a. speed

Angular Motion

- Same idea can be applied to angles
- Θ = angular position
- $d\Theta/dt$ is the derivative of angular position wrt to time
- $\omega = d\Theta/dt$, which is angular velocity
  - This is how fast the angular position is changing

Why does it matter?

- Linear velocity determines how fast the Duckiebot moves
- Angular velocity determines how fast the wheels spin
  - The faster the wheel spins, the faster the Duckiebot moves
- We can determine linear velocity based on angular velocity
  - $v = r*\omega$
- Using this relationship, we can calculate how far the Duckiebot has traveled
- Angular velocity also affects how the bot turns
- **THESE ASSUMPTIONS ONLY HOLD IF THERE IS NO SKIDDING ON THE TIRES NOR SIDEWAYS MOTION**
  - Important to note because real life does not work that way and real life is not perfect
  - Things happen in reality, such as defective tires

Instantaneous center of curvature (ICC)

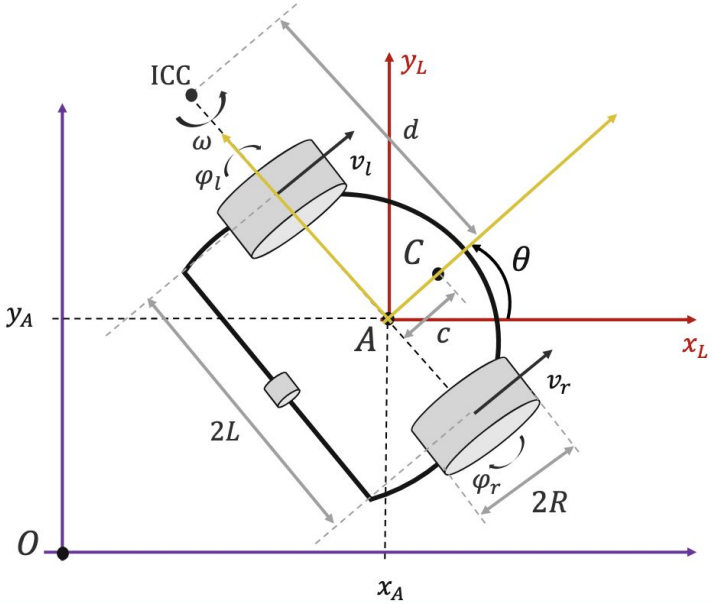
No slipping assumption → all points in a pure rotation field (centered at ICC) are subject to velocity orthogonal to the axle

$$\begin{cases} \dot{\theta}(d-L) = v_l \\ \dot{\theta}(d+L) = v_r \end{cases} \longrightarrow \begin{cases} d = L\dfrac{v_r + v_l}{v_r - v_l} \\ \dot{\theta} = \dfrac{v_r - v_l}{2L} \end{cases}$$

- Observations

1. If $v_r = v_l \Rightarrow$ no turn $\Rightarrow$ ICC is undefined
2. If $v_r = -v_l \Rightarrow$ turn "on itself" $\Rightarrow ICC \equiv A$
3. If $v_r = 0$ $(v_l = 0) \Rightarrow$ turn "on wheel"
   $\Rightarrow d = -L$ $(d = L)$

Moreover: $\dot{\theta} d = v_A$



ROS

- Robot Operating System
- Middleware
  - Communication channel between code and sensors
- Not an actual operating system
- To follow along, start at this URL
- https://github.com/duckietown/lx-ros-basics/blob/ente/notebooks/01_navigating_file_system.ipynb
- Might be easier to follow through a VM rather than Mac's approach of a container

Navigating ROS

- rospack
  - Used to find a package’s address
  - rospack find [package name]
- roscd
  - Similar to cd in command line, used to change to package directory
  - roscd [package name]
  - Works with subdirectories as well if you know the path
- rosls
  - Similar to ls, shows list of possible paths in a given package
  - rosls [package name]

Catkin Workspace

- src/
  - Add source code here for the packages you want to build
- build/
  - Where cache information + other intermediate files that come from compiling a workspace
- logs/
  - Log output, such as stdout or stderr, are kept here
- devel/
  - Where built artifacts are stored before installation
- install/
  - Where built artifacts are stored and executed from

Catkin Workspace

- To navigate and build into a workspace, execute
  - cd packages/
    catkin_make_isolated
  - Run this in /packages since this is the base directory
  - Alternatively, you can use “catkin make” or “catkin build”
- To activate the workspace, execute
  - source ./devel_isolated/setup.bash

Catkin Packages

- What does a Catkin package contain?
  - package.xml that contains metadata about the package
  - CMakeLists.txt that uses catkin
  - Only 1 package within the package
    - No nested packages nor multiple packages sharing the same directory

Catkin Packages

- Example on how to make a new package using dt-joystick-demo
- Should find dt-joystick-demo underneath packages
- Inside the package you should see package.xml with this inside

```
<package>
  <name>dt-joystick-demo</name>
  <version>0.0.1</version>
  <description>
  My first Catkin package in Duckietown.
  </description>
  <maintainer email="YOUR_EMAIL@EXAMPLE.COM">YOUR_FULL_NAME</maintainer>
  <license>None</license>

  <buildtool_depend>catkin</buildtool_depend>
</package>
```

Catkin Packages

- Replace the parts with YOUR NAME with your actual name and YOUR EMAIL with your email

```
<package>
  <name>dt-joystick-demo</name>
  <version>0.0.1</version>
  <description>
  My first Catkin package in Duckietown.
  </description>
  <maintainer email="YOUR_EMAIL@EXAMPLE.COM">YOUR_FULL_NAME</maintainer>
  <license>None</license>

  <buildtool_depend>catkin</buildtool_depend>
</package>
```

Catkin Packages

- CMakeLists.txt should also be under the dt-joystick-demo directory
- This is what the file should look like

```
cmake_minimum_required(VERSION 2.8.3)
project(dt-joystick-demo)

find_package(catkin REQUIRED COMPONENTS
  rospy
)

catkin_package()
```

Catkin Packages

- Rebuild workspace with
  - cd packages/
    catkin_make_isolated
- Activate workspace with
  - source ./devel_isolated/setup.bash
- To make sure the new package is recognized by catkin, run
  - echo $ROS_PACKAGE_PATH
- Output should look like this

```
/code/src/lx-ros-basics/packages/src/dt-joystick-demo:/code/src/dt-ros-commons/packages/duckietown:/code/src/dt-ros-commons/packages/duckietown_msgs:/code/src/dt-ros-commons/packages/duckietown_protocols:/opt/ros/noetic/share
```

- You should see dt-joystick-demo in the list

ROS Nodes

- What is a node?
  - Simple put, it is an executable file in the ROS package
- To communicate with other nodes, each node has a ROS client library to handle this
- Each node can publish or subscribe to a topic
  - Publish - nodes that broadcast information to a topic
  - Topic - communication channel where publishers send data to
  - Subscriber - nodes that receive information from a topic
- Communication can be done in Python or C++

ROS Nodes

- Open the graphical environment **ON TERMINAL** with
  - dts code vnc -R [robot name]
  - A link to a VNC environment should show up
  - Message looks like
    - VNC running at [link]
  - Open this link in Chrome
- To access the terminal on the online environment, use LXTerminal
- In the terminal, type the following command to get all active nodes
  - rosnode list

ROS Node

- rosnode info returns information about a specific node
  - rosnode info /[robot name]/[node name]
  - Same robot name as earlier
- It returns information about the node's publications, subscriptions, and services
- Example for the camera node

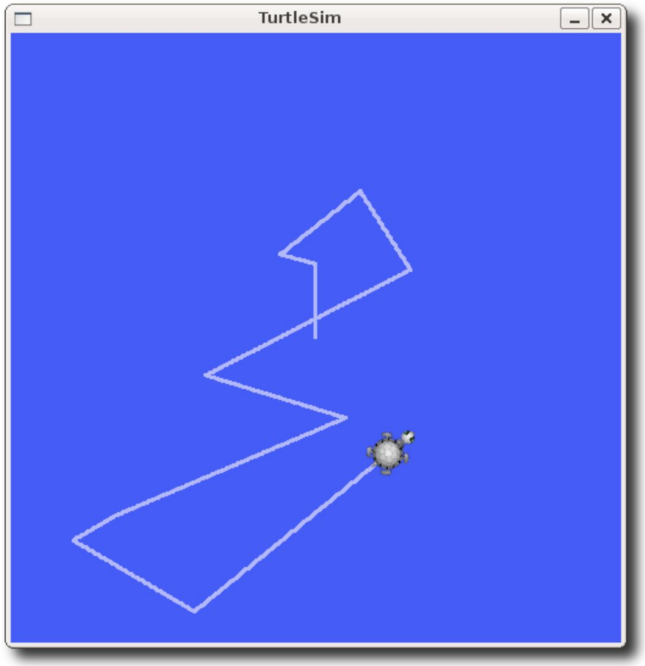
```
Node [/ROBOTNAME/camera_node]
Publications:
 * /rosout [rosgraph_msgs/Log]
 * /ROBOTNAME/camera_node/camera_info [sensor_msgs/CameraInfo]
 * /ROBOTNAME/camera_node/image/compressed [sensor_msgs/CompressedImage]

Subscriptions: None

Services:
 * /vlx/camera_node/get_loggers
 * /vlx/camera_node/get_parameters_list
 * /vlx/camera_node/request_parameters_update
 * /vlx/camera_node/set_logger_level
 * /vlx/camera_node/switch


contacting node http://ROBOTNAME.local:33271/ ...
Pid: 5538
Connections: ...
```

ROS Node

- rosrun allows you to use the package name to run a node without having to know the entire address
  - ex. rosrun turtlesim turtlesim_node
- To make it easier to revisit
  - rosrun turtlesim turtlesim_node __name:=my_turtle
- To ping a node, use rosnode ping [node name]
  - Ex. rosnode ping my_turtle

ROS Topic

- Re-open the web-based graphical interface if closed
  - Steps can be found on slide 23
- Re-open turtle sim if closed
  - rosrun turtlesim turtlesim_node
- Open a new terminal window
  - Run rosrun turtlesim turtle_teleop_key
  - Control the turtle with arrow keys on new terminal



rostopic subcommands

- rostopic -h to get a list of subcommands
- rostopic echo shows published data to a topic
  - usage is “rostopic echo [topic path]”
  - ex. rostopic echo /turtle1/cmd_vel shows the velocity of the turtle
  - On the right is an example of what would happen if the up arrow was pressed on the terminal where the turtle is controlled

```
linear:
  x: 2.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0
---
linear:
  x: 2.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0
---
```

rostopic subcommands

- rostopic type returns the type of the message being published
  - Usage is rostopic type [topic]
  - Ex. rostopic type /turtle1/cmd_vel returns geometry_msg/Twist
  - rosmsg show geometry_msg/Twist shows

```
geometry_msgs/Vector3 linear
  float64 x
  float64 y
  float64 z
geometry_msgs/Vector3 angular
  float64 x
  float64 y
  float64 z
```

rostopic subcommands

- rostopic pub publishes the messages to a topic
  - rostopic pub [topic] [msg_type] [args]
  - rostopic pub - 1 /turtle1/cmd_vel geometry_msgs/Twist -- ‘[2.0, 0.0, 0.0]’ ‘[0.0, 0.0, 1.8]’
  - This line publishes the message that tells modules subscribed to this publisher to move with a linear velocity of 2.0 in the x direction and an angular velocity of 1.8 in the z direction
  - -1 means publish only 1 message then exit
  - To repeat this command since this only runs once, change the command to rostopic pub /turtle1/cmd_vel geometry_msgs/Twist -r 1 -- ‘[2.0, 0.0, 0.0]’ ‘[0.0, 0.0, 1.8]’
  - Now this command will be ran at 1 Hz where 1 Hz = 1/sec
  - In other words, once every second

rostopic subcommands

- rostopic hz shows how often the data is being published
  - rostopic hz [topic]
  - Ex. rostopic hz /turtle1/pose shows the following message, which shows that the data is being published at roughly 60 Hz

```
subscribed to [/turtle1/pose]
average rate: 59.354
        min: 0.005s max: 0.027s std dev: 0.00284s window: 58
average rate: 59.459
        min: 0.005s max: 0.027s std dev: 0.00271s window: 118
average rate: 59.539
        min: 0.004s max: 0.030s std dev: 0.00339s window: 177
average rate: 59.492
        min: 0.004s max: 0.030s std dev: 0.00380s window: 237
average rate: 59.463
        min: 0.004s max: 0.030s std dev: 0.00380s window: 290
```

ROS Messages

- Messages are text files that contains the body of a ROS message
- Some field types include
  - int8, int16, int32, int64 and its uint counterparts
  - float32, float64
  - string
  - time, duration
  - msg files
  - Dynamic arrays and static length arrays
- The first line in a msg file contains a header
  - Contains timestamp and coordinate frame information

Creating ROS Message Example

- Create a new package by calling
  - catkin_create_pkg dt_joystick_demo_msgs rospy message_generation
- Inside the dt_joystick_demo_msg folder, go inside msg and there should be Num.msg
  - Should only contain int64 num
  - Can add more elements but only 1 per line like the image on the right

```
string first_name
string last_name
uint8 age
uint32 score
```

- Open package.xml in dt_joystick_demo_msg folder and unc
  <exec_depend>message_runtime</exec_depend>
- Open CMakeLists.txt in same folder and add/uncomment add_message_files(FILES Num.msg)
- Add the lines generate_messages() and catkin_package(CATKIN_DEPENDS message_runtime) in that order

Creating ROS Message Example

- In the terminal, call
  cd packages/
  catkin_make_isolated
- To check whether ROS can see the message, call
  rosmsg show Num
  To get the package name
- Use the package name and try
  rosmsg show [package name]/Num
  And that should show the contents of your message

Duckietown and ROS

- Build code with
  dts code build -R [robot name]
- Run code with
  dts code workbench -R [robot name] [-m]
  - Only call [-m] if Duckiematrix is running
  - If Duckiematrix isn’t running, call
    dts code start_matrix
- In a new terminal, call the following to open a noVNC desktop
  dts code vnc -R [robot name]